

Least Authority

PRIVACY MATTERS

Rabby Wallet Extension

Security Audit Report

Rabby Wallet

Final Audit Report: 9 October 2026

Table of Contents

- Overview
  - Background
  - Project Dates
  - Review Team
- Coverage
  - Target Code and Revision
  - Supporting Documentation
  - Areas of Concern
- Findings
  - General Comments
    - Code Quality
    - Documentation and Code Comments
    - Scope
  - Specific Issues & Suggestions
    - Issue A: Perps Agent Keys Remain in Memory After Wallet Lock
    - Issue B: Inconsistent Account and Operation Binding Across Signing Flows
  - Suggestions
    - Suggestion 1: Make Approval Type Explicit
    - Suggestion 2: Update Documentation and Code Comments
- About Least Authority
- Our Methodology

This Security Audit Report makes no statements or warranties and is for discussion purposes only.

Overview

Background

Rabby Wallet has requested that Least Authority perform a security audit of its Wallet Extension to assess developments since the previous review, for which the final audit report was delivered on September 2, 2025.

Project Dates

- **August 31, 2026 - September 9, 2026:** Initial Code Review *(Completed)*
- **September 11, 2026:** Delivery of Initial Audit Report *(Completed)*
- **October 8, 2026:** Verification Review *(Completed)*
- **October 9, 2026:** Delivery of Final Audit Report *(Completed)*

Review Team

- Frankie Pangilinan, Security Researcher and Engineer
- Dominic Tarr, Security Researcher and Engineer
- Burak Atasoy, Project Manager
- Jessy Bissal, Technical Editor

Coverage

Target Code and Revision

For this audit, we performed research, investigation, and review of the Rabby Wallet Extension followed by issue reporting, along with mitigation and remediation instructions as outlined in this report.

The following code repository was considered in scope for the review:

- Rabby Wallet Extension: https://github.com/RabbyHub/Rabby/compare/44c8e5...606cd9

Annual Audits will focus on the changes to the Rabby Wallet Extension repository since the previous audit. Specifically, the scope of this audit will be based on the differences (diffs) identified in the link provided above, with the following areas of focus:

- Checking that Rabby Wallet has not introduced functionality that could leak its private key mnemonic phrase outside the application.
- Considering potential interface-related security issues, such as those previously reported by whitehat researchers.

Specifically, we examined the Git following revision for our initial review:

- `606cd9c641a79aff335ee992ebe7fa8d8982c0bd`

For the verification, we examined the following Git revision:

- `21269dce6e41e49676cdab519a0a0b8289d9d7d8`

This Security Audit Report makes no statements or warranties and is for discussion purposes only.

For the review, this repository was cloned for use during the audit and for reference in this report:

- Rabby Wallet Extension: https://github.com/LeastAuthority/RabbyHub-Rabby-Wallet-Extension/tree/audit3

All file references in this document use Unix-style paths relative to the project's root directory.

In addition, any dependency and third-party code, unless specifically mentioned as in scope, were considered out of scope for this review.

Supporting Documentation

The following documentation was available to the review team:

- N/A.

In addition, this audit report references the following documents:

- Previous Security Audit Report by Least Authority on Rabby Wallet's Wallet Extension (PDF) *(delivered via email on 2 September 2025)*

Areas of Concern

Our investigation focused on the following areas:

- Correctness of the implementation;
- Adversarial actions and other attacks on the wallet;
- Attacks that impact funds, such as the draining or manipulation of funds;
- Mismanagement of funds via transactions;
- Malicious attacks and security exploits that would impact the wallet;
- Vulnerabilities in the wallet code and whether the interaction between the related network components is secure;
- Exposure of any critical or sensitive information during user interactions with the wallet and use of external libraries and dependencies;
- Leakage or exposure of the private key mnemonic phrase outside the application;
- Proper management of encryption and storage of private keys;
- Inappropriate permissions and excess authority;
- Data privacy, data leaking, and information integrity; and
- Anything else as identified during the initial analysis phase.

Findings

General Comments

Rabby Wallet is a non-custodial wallet for managing assets and interacting with decentralized applications on Ethereum and other EVM-compatible chains. Our review covered its browser extension, which incorporates MetaMask components, including a fork of its keyring controller.

We reviewed changes to key storage, wallet locking, onboarding, signing and approval flows, and the handling of sensitive data in extension messages. The main findings concern secret retention and signing state that remains active after its original account or request context changes. Since a vulnerability was recently publicly disclosed and subsequently remediated by the Rabby Wallet team, we also examined the underlying approval flow and suggest hardening it against similar attacks (Suggestion 1).

This Security Audit Report makes no statements or warranties and is for discussion purposes only.

The extension applies access controls to privileged operations. Its UI messaging handler rejects foreign extension IDs and senders identified as content scripts. Website origins are derived from browser-provided sender URLs. Provider methods subject to connection checks require connection approval for unconnected sites. Methods marked as safe are exempt from those checks.

The explicit private key and mnemonic export methods verify the supplied password before returning secrets. The GridPlus credentials field is also removed from unencrypted keyring records when saving state and restoring existing records at startup.

A configurable inactivity timer triggers wallet lock when enabled. Locking removes the main keyring service's password and keyring references, clears Manifest V3 session storage, and notifies connected applications and extension interfaces. This cleanup does not guarantee memory erasure or eliminate every secret held by other components.

The transaction approval screen requests simulation and evaluates security rules. Security results inform warnings and signing control availability, with options to acknowledge or ignore results.

Rabby Wallet is based on a fork of MetaMask, a long-established non-custodial wallet that relies on keeping the user's private keys secure on the device. While Rabby, like MetaMask, encrypts the keys, it uses a user-selected password strengthened with a key derivation function, PBKDF2. The key derivation parameters are intentionally weak, as they are in MetaMask, because the user's device may have limited computational resources, such as a standard laptop or phone, while an attacker could use highly parallelized hardware, such as a mining rig. Consequently, if an attacker gains access to the ciphertext containing the private keys and the user has not selected a high-entropy password, brute-forcing the password may be feasible.

It is therefore important to recognize that the encrypted ciphertext containing the private keys must itself be treated as sensitive data and protected from extraction. Although users are advised to select a high-entropy password, this is not enforced because users forgetting their passwords, and consequently losing access to their funds, presents a greater risk than funds being stolen through brute-forced passwords.

Dependencies

The project uses Warden and automated scanning of dependency changes to reduce supply-chain risk. We encourage continued monitoring of security advisories and package maintenance, alongside these existing controls.

Code Quality

The codebase separates background services, request controllers, and UI components. However, the wallet controller and transaction approval screen each handle many responsibilities. Smaller modules and generic key-storage code, with feature-specific logic kept separate, would make security-sensitive behavior easier to review and maintain.

Tests

The repository includes unit tests and some service-level integration tests. However, we found no tests covering the full flow from a website request through approval and signing.

Documentation and Code Comments

The `README` provides build instructions and an architecture overview, with separate notes on background services, UI pages, and transaction handling. However, these notes do not cover some features added since the previous audit, including Perps and MiniSign.

This Security Audit Report makes no statements or warranties and is for discussion purposes only.

In addition, code comments are generally sufficient, although some need updating to reflect the current behavior (Suggestion 2).

Scope

The scope of this review was sufficient and included all security-critical components.

Specific Issues & Suggestions

We list the issues and suggestions identified during the review in descending order of severity. In most cases, remediation of an issue is preferable, but mitigation is suggested as another option for cases where a trade-off could be required.

ISSUE / SUGGESTION	SEVERITY	STATUS
Issue A: Perps Agent Keys Remain in Memory After Wallet Lock	Medium	Partially Resolved
Issue B: Inconsistent Account and Operation Binding Across Signing Flows	Medium	Unresolved
Suggestion 1: Make Approval Type Explicit	Informational	Resolved
Suggestion 2: Update Documentation and Code Comments	Informational	Unresolved

Issue A: Perps Agent Keys Remain in Memory After Wallet Lock

Location

`src/background/service/perps.ts`

`src/background/controller/wallet.ts`

`src/background/controller/wallet.ts`

`src/ui/views/Perps/sdkManager.ts`

Synopsis

Locking Rabby leaves loaded Perps agent private keys in the background cache and in any initialized UI SDK signer. These keys remain accessible after lock, extending their exposure beyond the unlocked session. This conflicts with general security practices to minimize how long secrets remain in plaintext memory and restrict access when they are no longer needed.

Preconditions

For this issue to occur, the following must hold true:

- A Perps agent key must be loaded into the background cache or UI SDK, and the agent must remain authorized for a funded Hyperliquid account.
- The relevant background service or UI document must remain alive after lock, including while an earlier decryption completes.

This Security Audit Report makes no statements or warranties and is for discussion purposes only.

- The attacker must be able to execute code in a trusted extension UI context or read extension process memory.

Likelihood

Low.

Impact

High.

Severity

Medium.

Technical Details

On unlock, `unlockAgentWallets()` decrypts the persisted agent-vault map into `memoryState.agentWallets`. `getAgentWallet()` returns cached entries through UI-facing controller methods without checking the lock state. For agent signing, `initPerpsAgentAccount()` also passes the selected key to the module-global SDK instance.

`lockWallet()` clears the main keyring references and MV3 session storage but leaves the Perps cache and SDK signer intact. The background lock listener only stops a telemetry timer, while the UI lock listener navigates to `/unlock`. Neither invokes Perps cleanup, and `destroyPerpsSDK()` has no call sites in the audited source.

Pending decryption has no cancellation or lifecycle check before committing its results. Consequently, `unlockAgentWallets()` can populate the cache after lock, and an MV3 password-based decrypt can restore Perps session wrapping keys after session cleanup.

Mitigation

We suggest closing Rabby extension pages and restarting the extension or browser to end the contexts holding loaded keys and, if exposure is suspected, replacing or deregistering the affected Hyperliquid API wallet.

Remediation

We recommend clearing cached Perps keys and SDK signing credentials on lock, logout, and account changes, blocking key access while locked, and preventing pending operations from restoring secrets after cleanup. Agent signing should remain in the background to reduce plaintext copies in UI contexts. In addition, regression tests should cover these transitions, multiple UI documents, and operations completing after lock.

Status

The Rabby Wallet team has implemented changes that clear Agent caches and UI signing credentials and prevent stale asynchronous restoration after wallet lock. The issue is partially resolved because moving Agent signing entirely to the background was outside the scope of this patch and remains outstanding.

Verification

Partially Resolved.

This Security Audit Report makes no statements or warranties and is for discussion purposes only.

Issue B: Inconsistent Account and Operation Binding Across Signing Flows

Location

src/ui/views/Perps/sdkManager.ts

src/ui/views/Perps/hooks/usePerpsState.ts

src/ui/views/Perps/hooks/usePerpsPosition.ts

src/ui/views/Perps/hooks/usePerpsState.ts

src/ui/views/DesktopPerps/hooks/usePerpsProState.ts

src/ui/views/DesktopPerps/hooks/usePerpsProPosition.ts

src/ui/component/MiniSignV2/domain/ctx.ts

src/ui/component/MiniSignV2/state/SignatureManager.ts

src/ui/component/MiniSignV2/state/TypedDataSignatureManager.ts

src/background/controller/wallet.ts

src/ui/views/Perps/popup/OpenPositionPopup.tsx

src/ui/views/Perps/screen/home.tsx

src/ui/views/DesktopPerps/components/TradingPanel/components/TradingButtons.tsx

Synopsis

Perps and MiniSign do not consistently bind signing state to its originating account and request. Changing accounts or replacing a request can leave earlier work active, resulting in unintended actions, stale transaction parameters, or misdirected signatures.

Preconditions

For this issue to occur, the user must change accounts or start a new signing request while earlier work or transaction state remains active. This can occur if the Perps UI permits closing the order popup and navigating to account selection while submission is pending.

Likelihood

Low.

Impact

High.

Severity

Medium.

This Security Audit Report makes no statements or warranties and is for discussion purposes only.

Technical Details

The affected flows retain state across account or request changes without consistently checking ownership:

- **Perps:** `applyPerpsSigner()` replaces the signer on a shared exchange client. Pending operations can therefore continue with another account's signer. For seed-phrase and private key accounts, signing uses `wallet.signTypedData()` without another approval screen. This affects the leverage update/order sequence as well as delayed unified account and referral actions after approval. The order popup remains closable during submission, permitting navigation to the account selector within the same extension document. The order case requires switching before the leverage request succeeds. If the request is rejected or times out, submission stops.

- **Desktop Perps:** `withErrorHandler()` waits for approval handling, then runs trading and position management actions without rechecking the account. A switch window requires pending approval work. Trading buttons require enabling trading first, limiting order placement reachability. Exchange ownership checks may reject actions targeting the previous account's orders.

- **MiniSign preparation:** `buildFingerprint()` omits the sender and signer account and compares calldata length instead of contents. `ensureContext()` can combine a new configuration with earlier preparation. Sender validation may reject this combination.

- **MiniSign completion:** Signing continues after `close()`, and `resolve()` can deliver the earlier request's signatures to a newer request. Signing itself retains the original account.

Mitigation

We suggest waiting for pending operations to finish before switching accounts or starting another request. Closing a popup does not reliably cancel its work.

Remediation

We recommend binding each operation to its account, keyring type, chain, payload, and request ID. The operation should be invalidated on account changes, cancellation, or replacement, and its validity should be rechecked after asynchronous work and before signing, submission, or result delivery.

Status

The Rabby Wallet team has deferred remediation for this issue. The team noted that exploitation requires specific timing and workflow conditions and that remediation would require substantial changes across the Perps and MiniSign signing flows.

Verification

Unresolved.

This Security Audit Report makes no statements or warranties and is for discussion purposes only.

Suggestions

Suggestion 1: Make Approval Type Explicit

Location

RabbyHub/Rabby/commit/c300d2827ac668d9880411e28b050661daa0e8dd

Synopsis

A Rabby vulnerability was recently published by v12. The attack involves a malicious website issuing a signature request that would drain funds, while a UI bug hides the request from the user. The user is then distracted by a game. After a 10-minute delay, the wallet automatically locks. When the user unlocks it again, password entry calls `resolveApproval()` without arguments. This is interpreted as approval of the current pending request, which in this case is the malicious request.

Mitigation

The Rabby Wallet team addressed the vulnerability by adding a check on the unlock page to ensure that the current approval is an `Unlock` type. This resolves the specific attack because it relied on the unlock screen approving a request for a different action. However, `resolveApproval()` is used in many other flows, where it approves the current approval as the standard request-approval mechanism.

Remediation

We recommend hardening the approval flow against this attack pattern by refactoring `resolveApproval` to always take the expected approval type as an argument.

`resolveApproval` can currently be called with or without a data object and with additional optional parameters, such as the approval ID. The `forceReject` option, which is the third argument, is never used. Cases that use the approval ID call `getApproval()` first, which returns the current approval and makes the approval ID redundant.

The approval flow could also be hardened by adding a timeout. The attack could not have succeeded if pending approvals had expired after a few minutes.

Status

The Rabby Wallet team has implemented a more comprehensive fix for the vulnerability. Previously, the `id` parameter was optional and defaulted to the current approval, potentially causing the system to approve an unintended request. The updated implementation requires the `id` parameter for all calls to `resolveApproval`, with this requirement enforced through type checks. The original vulnerability has been resolved, and the updated interfaces prevent its reintroduction through the same mechanism.

Verification

Resolved.

Suggestion 2: Update Documentation and Code Comments

Location

`README.md`

`docs/background.md`

`docs/ui.md`

This Security Audit Report makes no statements or warranties and is for discussion purposes only.

docs/transaction.md

src/ui/app.tsx

src/ui/views/Perps/sdkManager.ts

src/ui/component/MiniSignV2/state/TypedDataSignatureManager.ts

src/background/service/keyring/index.ts

Synopsis

The README provides build instructions and an architecture overview, with separate notes on background services, UI pages, and transaction handling. These notes do not cover the Perps and MiniSignV2 flows added since the previous audit, and the UI notes still describe background-window access rather than the current messaging flow. The lock comment also describes deallocating all secrets without explaining the limits of that cleanup.

Mitigation

We recommend updating the architecture notes to cover Perps, MiniSignV2, and the current messaging flow. The scope of lock cleanup should also be clarified in code comments, and the effects of locking, cancellation, and account changes on pending operations and stored secrets should be documented.

Status

The Rabby Wallet team has deferred implementing this suggestion because its architecture has not yet been finalized. The team plans to update the README architecture diagram once the architecture changes are complete.

Verification

Unresolved.

This Security Audit Report makes no statements or warranties and is for discussion purposes only.

About Least Authority

Founded in 2011 and based in Berlin, Germany, since 2016, Least Authority provides security consulting for privacy-preserving technologies and decentralized systems. We are committed to transparency, openness, and advancing secure digital infrastructure.

We believe that people have a fundamental right to privacy and that secure solutions enable people to use the internet and other connected technologies more freely. Our security consulting services help teams make their solutions more resistant to unauthorized access and unintended system manipulation. We support teams from the design phase through production launch and beyond.

Our team has experience reviewing code for security vulnerabilities and specific attack vectors in multiple languages, including Rust, Go, Python, C, C++, Solidity, Cairo, Circom, Noir, Haskell, OCaml, TypeScript, Swift, ZoKrates, and others. We have reviewed implementations of cryptographic protocols and distributed system architecture in blockchains, payments, smart contracts, zero-knowledge protocols, and consensus protocols. Additionally, we use various tools to scan code and networks and build custom tools as necessary.

For more information about our security consulting, please visit:
https://leastauthority.com/security-consulting.

Our Methodology

We use a transparent review process that provides the project team visibility into our findings, assumptions, and remediation considerations throughout the audit. The goals of our security audits are to improve the quality of the systems we review and support sufficient remediation to help protect users. The following is the methodology we use in our security audit process.

Code Review

In manually reviewing all of the code, we identify potential issues, including those related to code logic, error handling, cryptographic implementations, and correctness with respect to specifications, standards, and best practices. We also evaluate areas where more defensive programming could reduce the risk of future mistakes and accelerate future audits. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

When appropriate, we also use artificial intelligence (AI) to support the efficiency of our security audits without reducing rigor or reliability. AI-assisted outputs are carefully guided, reviewed, and validated by our team so that conclusions remain accurate and defensible. This supports stronger audit decisions, not only faster review.

Vulnerability Analysis

Our audit techniques include manual code analysis, user interface interaction, and white box penetration testing, supplemented by various tools, including AI, that support the code review process. Discussions with the development team help clarify their vision for the software. Installing and using the relevant software helps us understand user interactions and roles, while also informing our analysis of threat models and attack surfaces. Design documentation, relevant papers, other audit results, similar projects, source code dependencies, and open issue tickets may also be reviewed to inform the analysis. We identify potential vulnerabilities, and, when appropriate, create issue entries. For each entry, we follow the issue investigation and remediation process described below.

This Security Audit Report makes no statements or warranties and is for discussion purposes only.

Documenting Results

We follow a conservative and transparent process for analyzing potential security vulnerabilities and supporting successful remediation. Whenever a potential issue is identified, we raise it with the client through a dedicated secure communication channel to discuss its validity and impact, to the extent these can be determined at that stage. This process allows potentially relevant findings to be reviewed with the client early, before they are documented in greater detail in the initial audit report. For each confirmed issue, we assess impact and feasibility and use those ratings to determine the overall severity, following the OWASP Risk Severity Matrix.

Suggested Solutions

We identify immediate and comprehensive mitigations available to live deployments and then suggest remediation requirements for future releases. When code is already deployed in production, we include mitigation strategies in the report to provide interim support until appropriate remediation is available. The mitigation and remediation recommendations should be reviewed by the development team and deployment engineers. After delivering the Initial Audit Report, we remain available to support implementation of the proposed mitigations and remediations before the verification review.

Development teams should address findings in an order and with an urgency that prioritizes the security of their users. Findings should be addressed as soon as possible, but remediation should be carried out pragmatically and should not cause unnecessary disruption to project timelines.

Although issues must be handled on a case-by-case basis, we work toward a resolution timeline that balances the impact on users and the needs of the project team.

Resolutions & Publishing

Once the findings are comprehensively addressed, we complete a verification review to assess whether the issues and suggestions have been sufficiently addressed. When this analysis is complete, we update the report and provide a Final Audit Report that can be published in full. If any critical issues remain, we recommend that the Final Audit Report not be published and that users and other stakeholders be alerted to the impact. We encourage all findings to be addressed and the Final Audit Report to be shared publicly to support transparency and advance security knowledge within the industry.

This Security Audit Report makes no statements or warranties and is for discussion purposes only.